

Agentic SRA Framework — Software Architecture Document (SAD)

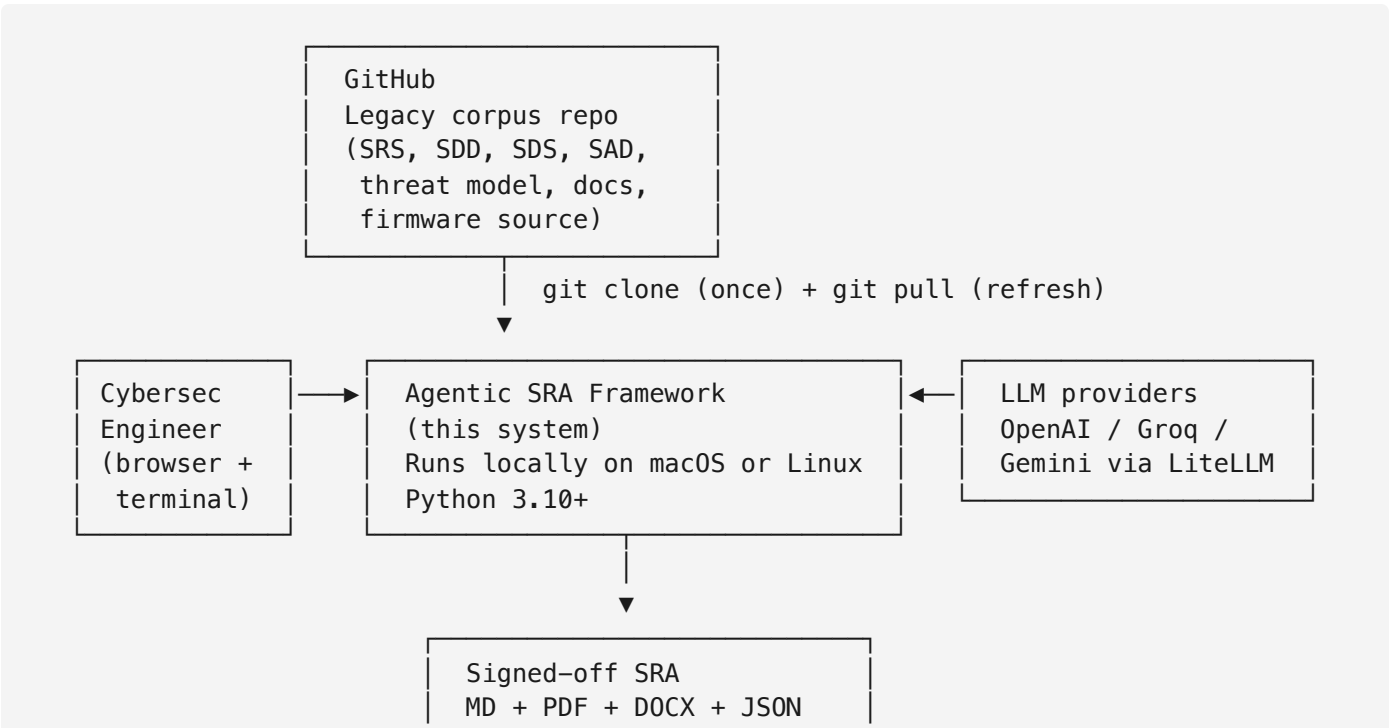
Document: SAD-FW-001 · **Version:** 1.0 · **Status:** Approved (2026-09-10) **Traces up to:** docs/requirements.md v1.0 **Traces down to:** docs/detail_design.md (Phase 4, pending)
Audience: Cybersecurity engineers, regulatory reviewers, capstone advisor

1. Purpose

This document describes the high-level architecture of the Agentic SRA Framework. It shows how the seven subsystems fit together to satisfy the 72 requirements in docs/requirements.md , what data flows between them, where trust boundaries lie, and which technology choices were made and why. The document does not specify class or function-level design — that lives in the Detail Design (Phase 4).

2. System context

The framework is a local-first CLI + local Streamlit UI that a single cybersecurity engineer runs on their own laptop.

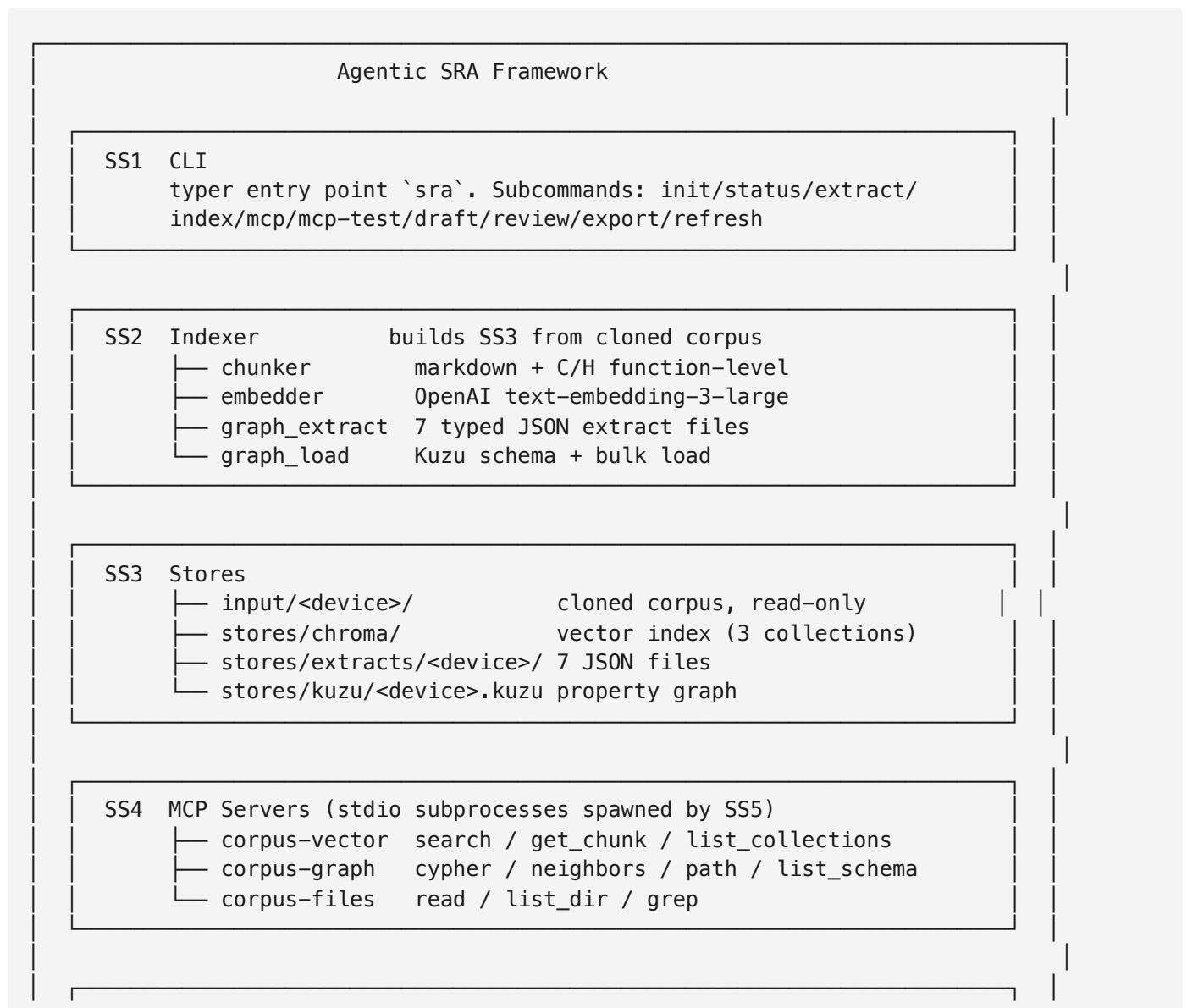


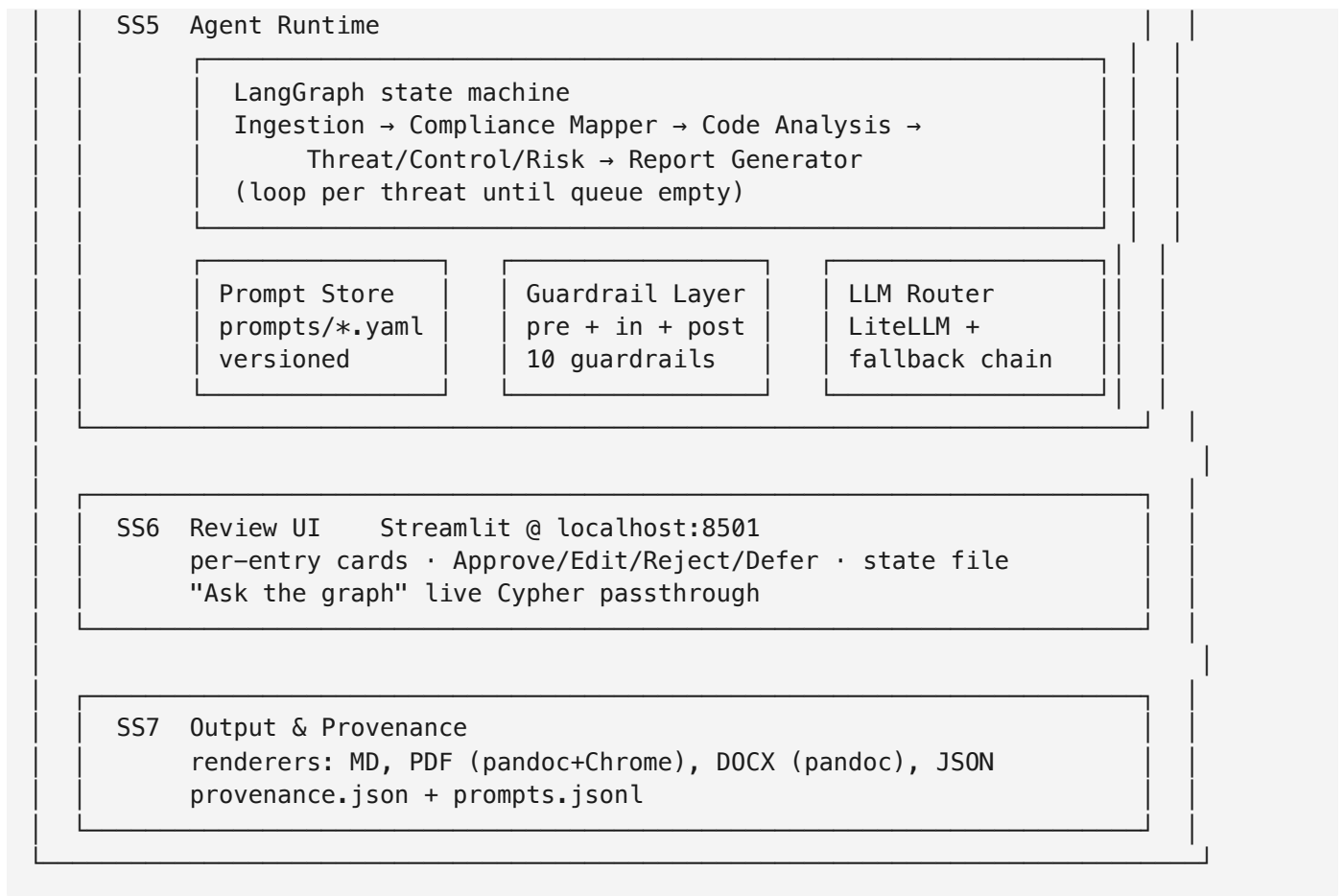
SAD-CTX-0001 · Single operator per device The framework serves one engineer working on one device at a time. Concurrent multi-user editing is explicitly out of scope for v1. *Traces to:* SRS-NFR-0006, out-of-scope §6.

SAD-CTX-0002 · Three external systems The framework interacts with exactly three external systems: GitHub (read-only, for cloning), the LLM providers (outbound calls for embeddings + agent reasoning), and the engineer's browser (rendering the review UI). *Traces to:* SRS-NFR-0006, SRS-CLI-0003, SRS-CLI-0011.

3. High-level architecture

Seven subsystems, one framework:





SAD-SYS-0001 · Seven subsystems The framework is divided into seven subsystems (CLI, Indexer, Stores, MCP Servers, Agent Runtime, Review UI, Output & Provenance). Each has a single responsibility and communicates with its neighbors through explicit contracts.

SAD-SYS-0002 · Two long-lived processes, many short-lived Only the Review UI (Streamlit) is a long-lived server process. Everything else is short-lived per invocation of a CLI command. MCP servers are spawned as subprocesses by the Agent Runtime for the duration of a `sra draft` run and torn down at exit.

SAD-SYS-0003 · One-way dataflow between subsystems

- Indexer reads Cloned Corpus, writes to Stores
- Agent Runtime reads Stores through MCP Servers, writes to Output
- Review UI reads Output + Stores, writes to Output (final)

No subsystem writes back to Cloned Corpus or reads directly from another subsystem's private files.

4. Data flow (end-to-end)



5. Subsystem responsibilities

5.1 SS1 — CLI

SAD-CLI-0001 · Single Typer entry point The `sra` command is a single Typer app with subcommands. All commands share a common `--device` argument and read paths from `framework/config.py`. *Realizes*: SRS-CLI-0001, SRS-CLI-0002.

SAD-CLI-0002 · CLI does no business logic The CLI is a thin adapter — each subcommand parses arguments, imports the relevant subsystem module lazily, calls one function, and formats the return for the terminal. *Rationale*: keeps startup fast, makes each subsystem independently testable.

5.2 SS2 — Indexer

SAD-IDX-0001 · Four-stage pipeline Indexing runs in fixed order: chunk → embed → extract → load. Each stage writes to disk before the next starts, so partial failures leave the store in a defined state. *Realizes*: SRS-CLI-0007, SRS-CLI-0008.

SAD-IDX-0002 · One chunker per artifact family Six parsers live in `chunker.py` : requirement-like docs (SRS/SDD/SDS/SAD), vulnerability plan, threat catalog, narrative markdown, and C/C++ code. Each returns a list of `Chunk(id, text, metadata)` . *Realizes*: SRS-IDX-0004, SRS-IDX-0005.

SAD-IDX-0003 · Deterministic and idempotent Chunk ids are content-derived and stable across runs. The graph loader wipes and rebuilds the Kuzu database, so re-running `sra index` on the same corpus commit produces identical stores. *Realizes*: SRS-IDX-0005, SRS-IDX-0010, SRS-NFR-0010.

5.3 SS3 — Stores

SAD-STO-0001 · Chroma vector store Chroma is used as an embedded persistent client at `stores/chroma/` , with three collections: `device_docs` , `device_code` , `regulatory` . The Regulatory collection is loaded in a separate CLI step (`sra index --regulatory` , added in Phase 5 alongside the regulatory corpus). *Realizes*: SRS-IDX-0001, SRS-IDX-0003.

SAD-STO-0002 · Kuzu property graph Kuzu is used as an embedded database at `stores/kuzu/<device>.kuzu` . Schema is 11 node tables and 8 relationship tables, loaded from the seven JSON extract files. *Realizes*: SRS-IDX-0008.

SAD-STO-0003 · Extracts are the interchange format JSON extract files under `stores/extracts/<device>/` are the shared interchange format between the Indexer and the Graph Loader. They are human-readable, git-friendly, and re-loadable, and they carry `_dangling_edges.json` as a first-class artifact so documentation gaps are visible. *Realizes*: SRS-IDX-0007, SRS-IDX-0009.

5.4 SS4 — MCP Servers

SAD-MCP-0001 · Three servers, all stdio Each MCP server is a standalone Python module launched over stdio. The Agent Runtime spawns them once per `sra draft` invocation and communicates via the MCP protocol. *Realizes*: SRS-MCP-0007.

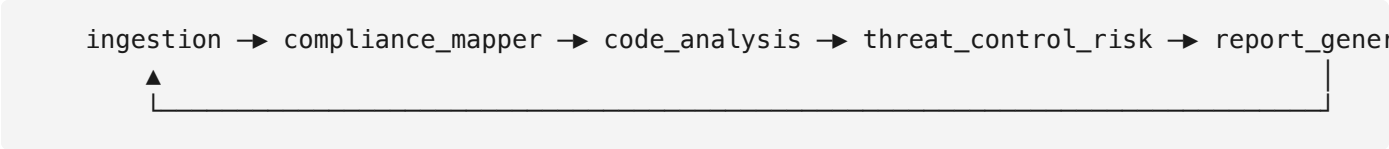
SAD-MCP-0002 · MCP as tool boundary Agents never import Chroma or Kuzu directly. Every store access goes through MCP. This isolates the agent code from backend choice, makes external clients (Claude Code, Cursor) able to reuse the same tools, and gives us one place to enforce guardrails 2, 4, and 5 (read-only, path safety, size caps). *Realizes*: SRS-MCP-0001, SRS-MCP-0002, SRS-MCP-0003, SRS-MCP-0004, SRS-MCP-0005.

SAD-MCP-0003 · No writes from agents to stores The `corpus-graph` MCP server refuses write operations. There is no write-capable store MCP; if we ever need one it will be a separate server so an operator can enable it explicitly. *Realizes*: SRS-MCP-0003.

5.5 SS5 — Agent Runtime

SAD-AGT-0001 · LangGraph state machine Agents run inside a LangGraph state graph. State is a Pydantic model carrying: device, threat queue, current threat, per-threat findings (from Compliance + Code Analysis), current SRA entry, tool-call log.

Nodes:



Loop terminates when the threat queue is empty. *Realizes:* SRS-AGT-0001..0005.

SAD-AGT-0002 · Prompt Store subcomponent Prompt templates live at `framework/agents/prompts/*.yaml`. Each YAML carries: version, model, temperature, system prompt, few-shot examples, output JSON schema, in-prompt guardrails, and a `guardrails_post` list naming which post-validators to run. *Realizes:* SRS-AGT-0006, part of failure-mode guardrails #6, #9, #13, #14.

SAD-AGT-0003 · Guardrail Layer subcomponent Ten guardrails are enforced across four layers (pre, in-prompt, tool, post) — see §5.5.1 below. *Realizes:* SRS-AGT-0007, SRS-AGT-0009.

SAD-AGT-0004 · LLM Router subcomponent LiteLLM is the only path to any LLM provider. The Router reads the `--profile` flag (SRS-CLI-0016), picks per-agent models from SRS-AGT-0011, and manages the fallback chain: on 429/timeout retry up to 3 times with exponential backoff, then spill to OpenAI `gpt-4o-mini`. Every call is logged to `prompts.jsonl` regardless of success/failure/fallback. *Realizes:* SRS-AGT-0006, SRS-AGT-0007, SRS-AGT-0008, SRS-AGT-0011, SRS-NFR-0005.

SAD-AGT-0005 · Agents cross-verify The Threat/Control/Risk agent is the only agent that produces the final 12-field SRA entry, but it may only assert findings that Compliance Mapper or Code Analysis has already verified. This makes cross-verification a structural property, not a prompt-level plea. *Realizes:* failure-mode guardrail #2 (cross-agent).

5.5.1 Guardrail Layer — the ten guardrails

#	Guardrail	Layer	Where it lives
1	Doc-ID citation → verify against Kuzu	Post	<code>guardrails/citation_verify.py</code>

#	Guardrail	Layer	Where it lives
2	Code claim → grep-verified via Code Analysis	Cross-agent	Enforced in Agent Runtime state machine
3	CVSS vector → parse + score sanity check	Post	<code>guardrails/cvss_check.py</code>
4	Regulatory quote → substring-verify against retrieved chunk	Post	<code>guardrails/quote_check.py</code>
6	Prompt injection → <code><CORPUS_UNTRUSTED></code> delimiters	Pre + In-prompt	Chunk formatter + system prompt
7	Per-entry token cap (10 k in / 4 k out)	Post	LLM Router enforcement
9	Legacy-vs-modern control taxonomy	In-prompt	Every relevant agent prompt YAML
11	JSON schema + retry-with-correction	Tool + Post	LLM Router (JSON mode) + <code>guardrails/schema_check.py</code>
13	Deprecated-crypto blocklist	In-prompt + Post	Prompt + <code>guardrails/banned_crypto.py</code>
14	In-prompt scope statement	In-prompt	Every agent prompt YAML

Hard-fail guardrails: 1, 3, 4, 11 — validator failure marks entry `parse_error` and preserves raw LLM output for manual review. **Soft-warn** guardrails: 9, 13 — entry flows through with `warnings: [...]` populated. *Realizes*: the 10 must-have guardrails locked in Phase 2 discussion.

5.6 SS6 — Review UI

SAD-UI-0001 · Streamlit single-page app The Review UI is a single `app.py` running on Streamlit at `localhost:8501`. It reads `output/<device>_sra_draft.json` on load and writes decisions to `output/<device>_review_state.json` on every action. *Realizes*: SRS-UI-0001, SRS-UI-0005.

SAD-UI-0002 · UI-to-graph live query The "Ask the graph" action makes an in-process Cypher query directly against the Kuzu store (not via MCP subprocess) for latency. Read-only queries

only, same regex block as the MCP server. *Realizes:* SRS-UI-0006.

5.7 SS7 — Output & Provenance

SAD-OUT-0001 · Four output formats, one source of truth The `.json` file is the source of truth. MD is rendered deterministically from it; PDF and DOCX are rendered from MD via pandoc. *Realizes:* SRS-OUT-0001, SRS-OUT-0002, SRS-OUT-0003, SRS-OUT-0004.

SAD-OUT-0002 · Provenance recorded per entry, per LLM turn `provenance.json` records per-entry: models used, prompt version, guardrail results, review decision, reviewer, timestamps. `prompts.jsonl` records one line per LLM turn with the exact prompt, tool calls, response, latency, and tokens. *Realizes:* SRS-OUT-0005, SRS-OUT-0006, SRS-NFR-0009.

6. Cross-cutting concerns

SAD-XCT-0001 · Logging All framework modules use Python `logging` with a shared configuration in `framework/log.py`. Level defaults to `INFO`, `--verbose` raises to `DEBUG`. Nothing sensitive (API keys, full corpus content) is logged. *Realizes:* SRS-NFR-0007.

SAD-XCT-0002 · Configuration Filesystem paths and framework-wide defaults live in `framework/config.py` and read from `.env`. There is no `config.yaml` in v1 — model plan lives in `framework/agents/prompts/*.yaml` per agent and is overridden by the `--profile` flag. *Rationale:* one less file to explain; per-agent YAML is a natural home for model/temperature.

SAD-XCT-0003 · Error handling posture Framework-level errors (missing input, invalid CLI args) exit with a non-zero code and a clear message. Agent-level errors (LLM failure after retries+fallback) mark the affected entry with `parse_error` but let the pipeline continue. *Realizes:* SRS-AGT-0009, SRS-NFR-0013.

SAD-XCT-0004 · Secrets API keys are read from `.env` at startup via `python-dotenv`. `.env` is `.gitignore`d. Keys never appear in logs, in provenance, in prompts, or in outputs. *Realizes:* SRS-NFR-0007.

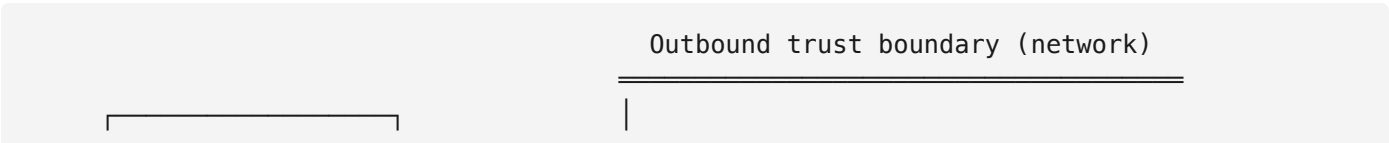
SAD-XCT-0005 · Reproducibility The framework records: corpus commit SHA, framework git commit SHA, model ids and versions, prompt versions, `--profile`, `temperature=0`. A re-run of `sra draft` on the same commit with the same profile should produce equivalent entries. *Realizes:* SRS-NFR-0004.

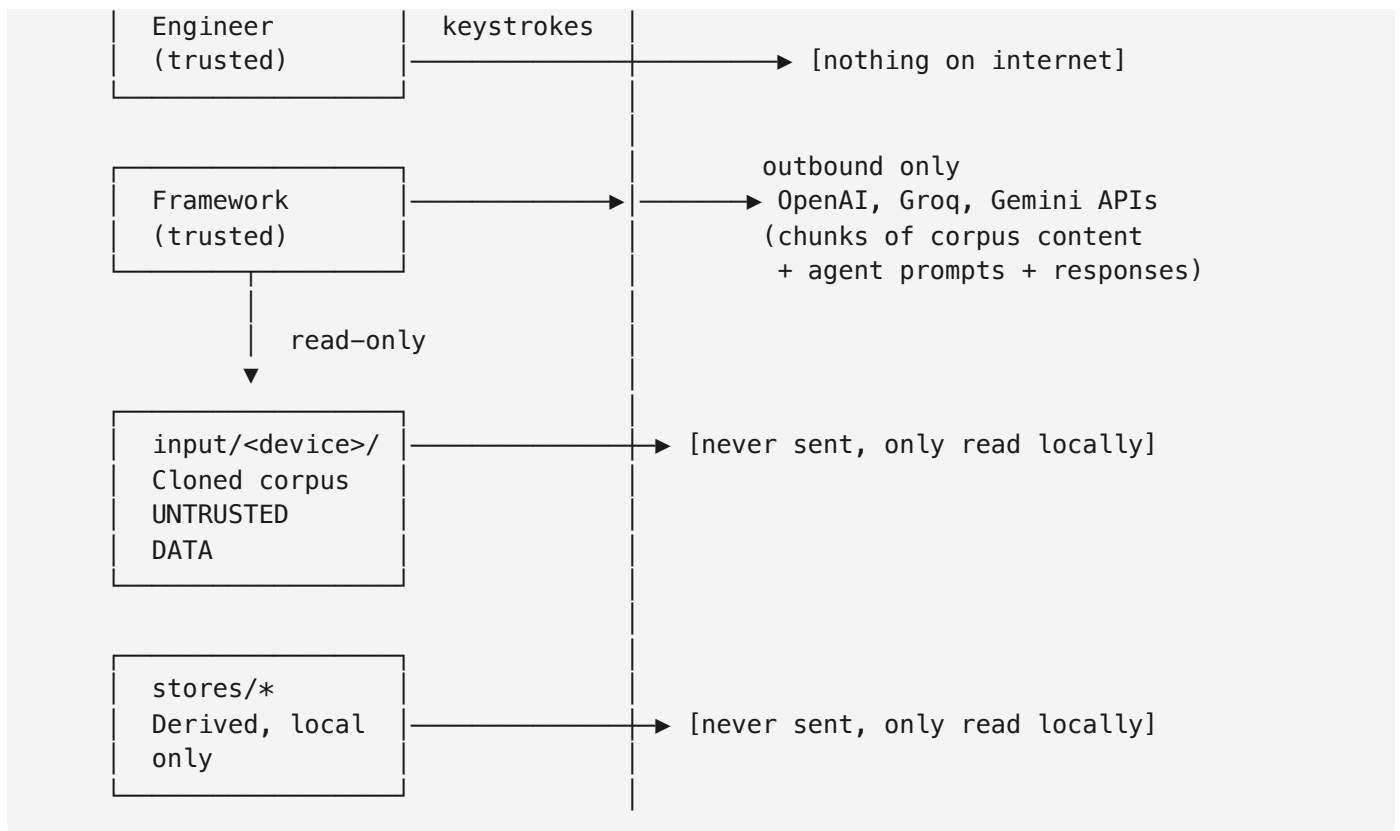
7. Technology choices

Choice	Alternatives considered	Rationale
Chroma for vectors	Qdrant (needs server), pgvector (needs Postgres), FAISS (no metadata filter)	Embedded, <code>pip install</code> , file-backed, zero setup; good enough at capstone scale
Kuzu for graph	Neo4j (server), Memgraph (server), NetworkX (in-memory only)	Embedded Cypher-like DB, no server, git-friendly file, reproducible on any laptop
LangGraph for orchestration	CrewAI (role-based, less control), raw asyncio (no state machine)	Explicit state machine, industry-standard, good tracing
LiteLLM for LLM router	Direct provider SDKs, langchain-openai	Multi-provider by config, needed for the <code>--profile</code> flag
MCP for tool boundary	Direct Python function calls	Backend-agnostic, reusable from other clients (Claude Code, Cursor), aligned with 2026 industry practice
Streamlit for review UI	FastAPI+React, Gradio, Textual	Single-file, no build step, capstone-appropriate, engineer sees a real UI
OpenAI text-embedding-3-large	Local <code>bge-large-en-v1.5</code> , Gemini embeddings	Best quality; \$0.05 one-time cost is negligible; frees the free-tier budget for agents
pandoc + Chrome headless for PDF	weasyprint (SIP blocks it on macOS), wkhtmltopdf (unmaintained)	Deterministic, works on macOS + Linux, high-quality output
Typer for CLI	Click, argparse	Typing-driven, auto-generates <code>--help</code> , clean subcommand structure

8. Trust boundaries and the framework's own DFD

The framework has three trust boundaries:





SAD-TB-0001 · Cloned corpus is untrusted data Content inside `input/<device>/` is treated as data, not as instructions. Every retrieved chunk is wrapped in `<CORPUS_UNTRUSTED>...</CORPUS_UNTRUSTED>` tags in prompts, and the system prompt states that content inside those tags is never to be followed as directives. *Realizes*: guardrail #6.

SAD-TB-0002 · Only three outbound flow types The only data that leaves the machine is: (a) embedding batches to OpenAI, (b) agent prompts + tool results to LLM providers, (c) `git pull` metadata to GitHub. Nothing else — no telemetry, no analytics, no cloud stores. *Realizes*: SRS-NFR-0006.

SAD-TB-0003 · Reference corpus is synthetic The PMx-100 corpus contains no real PHI. This lets us skip PHI-scrubbing in v1 without accepting real risk, and it lets us commit the corpus publicly. The Detail Design (Phase 4) will note where PHI scrubbing would be added for a real deployment. *Realizes*: deferred failure-mode #5.

9. Assumptions and constraints

#	Assumption / Constraint	Impact
A1	Engineer's laptop has network access to GitHub + LLM providers	If offline, only cached agents that hit no free-tier limit can run — realistically, must be online

#	Assumption / Constraint	Impact
A2	Legacy corpus follows the layout corpus/<device>/*.md + `firmware/**/*.c	*.h`
A3	Free-tier providers stay available with roughly current terms (Groq gpt-oss-120b, Gemini 2.5 Flash)	If a provider disappears, the <code>--profile free</code> runs by spilling to OpenAI (SRS-AGT-0007)
A4	OpenAI embedding pricing stays within an order of magnitude of \$0.13/M tokens	If it 10× we revisit the embedder choice
A5	Reviewer is one person	Multi-reviewer concurrent editing is a v2 concern
A6	Corpus is committed to Git	Provenance depends on immutable commit SHAs

10. Traceability — SAD → SRS

SAD section	Realizes SRS requirements
§2 System context, §5.1 CLI	SRS-CLI-0001, 0002, and all subcommand requirements
§5.2 Indexer	SRS-CLI-0007, 0008, SRS-IDX-0002, 0004-0007, 0010
§5.3 Stores	SRS-IDX-0001, 0003, 0008
§5.4 MCP Servers	SRS-MCP-0001 through 0007
§5.5 Agent Runtime	SRS-AGT-0001 through 0011, SRS-NFR-0004, 0005
§5.5.1 Guardrail Layer	10 must-have guardrails from Phase 2
§5.6 Review UI	SRS-UI-0001 through 0007
§5.7 Output & Provenance	SRS-OUT-0001 through 0007, SRS-NFR-0009
§6 Cross-cutting	SRS-NFR-0007, 0010, 0013
§7 Technology choices	SRS-NFR-0005, 0008, 0014
§8 Trust boundaries	SRS-NFR-0006, 0007, guardrails #5 (deferred) and #6

No SRS requirement is left un-realized. Every SAD paragraph traces up to at least one SRS item.

11. Out of scope for v1

- Multi-device concurrent indexing.
- Multi-user concurrent review (locking, presence, conflict resolution).
- Web-hosted deployment behind auth.
- Continuous re-indexing triggered by GitHub webhooks.
- Direct integration with the FDA eSTAR portal.
- SBOM ingestion (v1 reads code + docs; SBOMs added later).
- Fuzzing / dynamic analysis of firmware.
- PHI scrubbing (corpus is synthetic; document where the hook goes).
- Prompt A/B testing framework (belongs to Phase 6 evaluation).

12. Reviewer decisions (locked v1.0)

Question	Decision
Regulatory corpus in v1 scope	Yes — all four: FDA 2023 Cybersecurity in Medical Devices, IEC 62443-4-2 (Component Security Requirements), AAMI TIR57 (Principles for Medical Device Security), NIST SP 800-193 (Platform Firmware Resiliency)
Kuzu per-device vs single DB	Per-device (<code>stores/kuzu/<device>.kuzu</code>) — clean isolation, simple refresh, no cross-device coupling
Review UI concurrency	Hard constraint: one reviewer at a time per device. Documented in <code>README.md</code> and shown in the review UI on startup. No file lock in v1.
Regulatory corpus location	In-tree JSON at <code>framework/regulatory/*.json</code> under the framework repo — versioned with the framework, no extra clone step, editable for future clause additions

13. Change log

- **v1.0 (2026-09-10)** — Approved. Locked four reviewer decisions on regulatory corpus scope, Kuzu file layout, review UI concurrency, and regulatory corpus location.
- **v0.1 (2026-09-10)** — Initial draft. 7 subsystems, 10 guardrails, 8 trust-boundary flows, complete SRS coverage.